What is an Algorithm ?

- An “ALGORITHM” is a finite set of instructions that, if followed, accomplishes a particular task.
- In addition, all algorithm must satisfy the following criteria:
    1. **Input** – Zero or more quantities are externally supplied
    2. **Output** – At least one quantity is produced
    3. **Definiteness** – Each instruction is clear and unambiguous
    4. **Finiteness** – After tracing all instructions in ALG, the ALG terminates after a finite number of steps
    5. **Effectiveness** - Every instruction must be very basic so that it can be carried out, in principle, by a person using only pencil and paper. Also each operation must feasible

Algorithm Specification Pseudocode Conventions:

- ALG can be described using many ways
- Graphic representation called flowcharts are another possibility, but they work well only if the algorithm is small and simple
- In this text we present most of our algorithm using a Pseudocode that resembles C program
    1. Comments begins with // and continue until the end of line
    2. Blocks are indicated with matching braces { and }
    3. An identifier begins with a letter. The data types of variables are not explicitly declared
    4. Assignment of values to variables is done using the assignment statement - <variable> = <expression>;
    5. Boolean values – true and false, logical operators – and, or and not, relational operators - <, >, == etc. are provided
    6. Elements of multidimensional arrays are accessed using [and]. Array indices start at zero
    7. for, while and repeat-until looping statements are provided-

```
while < condition> do    for variable:= value1 to value2 step step do
{                        {
    statement 1;             statement 1;
      ….                       …….
    statement n;             statement n;
}                        }
```

A **repeat-until** statement is constructed as follows-

```
repeat
<statement 1>
          ….
<statement n>
until<condition>
```

8. A conditional statements has the following forms-

   ```
   if <condition> then <statement>
   if <condition> then <statement1> else <statement2>
   case
   {
       :<condition1>:  <statement1>;
           ……
           ……
       :<condition n>:  <statement n>;
       :else: <statement n + 1>
   }
   ```

9. Input and Output are given using the instruction read and write
10. An algorithm consists of a heading and a body – Algorithm Name (<parameter list>)

Example:- ALG to find maximum element in Array

```
1  Algorithm Max(A, n)
2  // A is an array of size n.
3  {
4      Result := A[1];
5      for i := 2 to n do
6          if A[i] > Result then Result := A[i];
7      return Result;
8  }
```

Algorithm to find GCD of two numbers:

ALGORITHM *Euclid(m, n)*

//Computes gcd(m, *n) by Euclid's algorithm*

!!Input: Two nonnegative, not -both-zero integers *m and n*

//Output: Greatest common divisor of *m and n*

while *n ≠ 0 do*

r ← *m* ***mod n***

m ← *n*

n ← *r*

return *m*

Fundamentals of Algorithmic Problem Solving

- Understanding the Problem
- Ascertaining the Capabilities of the computing device
- Choosing between Exact and Approximate Problem Solving
- Algorithm Design Techniques
- Designing an Algorithm and Data Structures
- Methods of Specifying the Algorithms
- Proof for Algorithmic Correctness
- Analysis of Algorithms
- Coding an Algorithm
- Program Testing

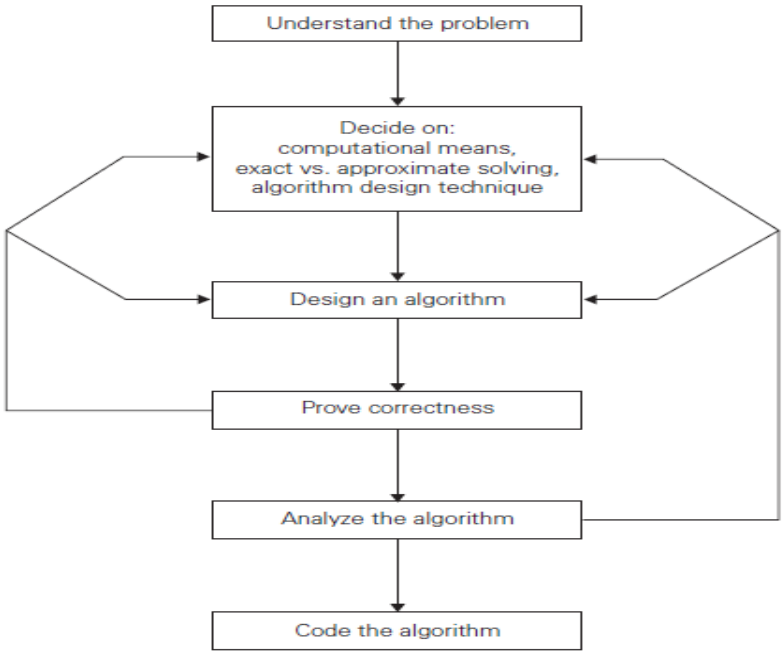


FIGURE 1.2 Algorithm design and analysis process.

Understanding the Problem :

- An input to an algorithm specifies an instance of the problem the algorithm solves.
- It is very important to specify exactly the set of instances the algorithm needs to handle.
- If you fail to do this, your algorithm may work correctly for a majority of inputs but crash on some “boundary” value.
- The designer should specify the assumptions made to solve this problem and the task to be carried out to achieve the result.

Ascertaining the capabilities of a computational device :

- Once we understand the problem clearly we need to ascertain the capabilities of the device.
- The vast majority of algorithms in use today are still destined to be programmed for a computer closely resembling the von Neumann machine.
  - Sequential Algorithms : The algorithms that are designed to be executed Von Neumann architecture(sequentially) are called Sequential Algorithms.
  - Parallel Algorithms: The algorithms that are designed to be executed on parallel computers are called Parallel Algorithms.

Choosing between Exact and Approximate Problem Solving :

- The next principal decision is to choose between solving the problem exactly or solving it approximately.
- For certain problems, we will get the exact result. So we must choose between an algorithm which gives exact result(often called exact algorithm) and an algorithm that gives approximate result(often called approximate algorithms).

Algorithm Design Techniques :

- An algorithm design technique (or “strategy” or “paradigm”) is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.
- Learning these techniques is of utmost importance for the following reasons.
  - First, they provide guidance for designing algorithms for new problems, i.e., problems for which there is no known satisfactory algorithm.
  - Second, algorithms are the cornerstone of computer science. Every science is interested in classifying its principal subject, and computer science is no exception.
- Algorithm design techniques make it possible to classify algorithms according to an underlying design idea; therefore, they can serve as a natural way to both categorize and study algorithms.

Designing an Algorithm and Data Structures :

- While the algorithm design techniques do provide a powerful set of general approaches to algorithmic problem solving, designing an algorithm for a particular problem may still be a challenging task.
- Once we have selected the type of the algorithm then we have to choose the proper data structures to represent the various data types or for easy manipulation so as to achieve the result.

Methods of Specifying an Algorithm :

- Once you have designed an algorithm, you need to specify it in some fashion.
- These are the two options that are most widely used nowadays for specifying algorithms.
  - Using a natural language has an obvious appeal; however, the inherent ambiguity of any natural language makes a succinct and clear description of algorithms surprisingly difficult.
  - Pseudocode is a mixture of a natural language and programming language like constructs. Pseudocode is usually more precise than natural language, and its usage often yields more succinct algorithm descriptions.

Proof for Algorithm’s Correctness :

- After specifying the algorithm for a specific problem, we have to prove its correctness.
- It is our responsibility to prove that the algorithm produces the required output for every legitimate input in a finite amount of time.

- For some algorithms, a proof of correctness is quite easy; for others, it can be quite complex.
- A common technique for proving correctness is to use mathematical induction because an algorithm's iterations provide a natural sequence of steps needed for such proofs**.**

Analysis of Algorithms :

- After correctness, by far the most important is efficiency.
- In fact, there are two kinds of algorithm efficiency:
  - Time efficiency indicates how fast an algorithm runs.
  - Space efficiency indicates how much extra memory is required for the algorithm.
- Another desirable characteristic of an algorithm is simplicity.
- Yet another desirable characteristic of an algorithm is generality. There are, in fact, two issues here: generality of the problem the algorithm solves and the set of inputs it accepts.

Coding an Algorithm :

- Most algorithms are destined to be ultimately implemented as computer programs.
- Implementing an algorithm correctly is necessary but not sufficient.
- Modern compilers do provide a certain safety net in this regard, especially when they are used in their code optimization mode.
- The algorithm designed should be implemented using a programming language such as C, C++ or any other suitable language.
- Usage of different languages for solving a problem results in different memory requirements and affects the speed of the program.
- The language selected should support the features mentioned in the design phase.

Program Testing :

- After implementing the algorithm in a specific language, next is the time to execute.
- Testing is nothing but the verification of the program for its correctness.
- Any logical error can be identified by program testing.
- Debugging is part of testing.

Analysis Framework

- The efficiency of an algorithm can be decided by measuring the performance of an algorithm.
- The performance of an algorithm can be measured by computing two factors:

- Amount of time required by an algorithm to execute.
- Amount of storage required by an algorithm.

Analysis Framework :

- There are two kinds of efficiency – Time and Space
- Time efficiency indicates how fast an algorithm in question runs
- Space efficiency deals with the extra space the algorithm requires
- Measuring an Input's size – almost all ALGs run longer on larger inputs. For example, it takes longer to sort larger arrays, multiply larger matrices, and so on
- Units for Measuring Running Time – measuring an ALGs running time, simply can use a standard unit as seconds, milliseconds and so on. But, practically to identify the most important operation of the algorithm, called the basic operation, the operation contributing the most to the total running time, and compute the number of times the basic operation is executed. Hence, we can estimate, $T(n) \approx C_{op}C(n)$, where T(n) is running time of a program, C_{op} be the execution time of an ALGs basic operation on a particular computer, and C(n) be the number of times this basic operation needs to be executed for this algorithm.
- Worst-Case Efficiency –It is an efficiency when algorithm runs for a longest time. Example – $C_{worst}(n)=n$
- Best-Case Efficiency – It is an efficiency when the algorithm runs for short time. Example – $C_{best}(n)=1$
- Average-Case Efficiency – This type of efficiency gives information about the behaviour of an algorithm on specific or random input. Example – $C_{avg}(n)=p(n+1)/2 + n(1-p)$

Performance Analysis

- Major goal of this subject is to develop skills for making evaluative judgments about algorithm
- There are many criteria upon which we judge algorithm-
    1. Does it do what we want it to do?
    2. Does it work correctly according to the original specification of the task?
    3. Is there documentation that describes how to use it and how it works?
    4. Are procedures created in such a way that they perform logical sub-functions?
    5. Is the code readable?
- There are other important criteria for judging algorithms that have a more direct relationship to performance
    1. Space Complexity (Storage Requirement)
    2. Time Complexity (Computing Time)

Space Complexity (Storage Requirement)

- The space complexity of an algorithm is the amount of memory it needs to run to completion
- The space needed is the sum of a fixed part and variable part
    1. Fixed Part – is independent of characters of the inputs and outputs. It includes instruction space(space for code), space for simple variables and fixed size component variables, space for constants etc.
    2. Variable Part – consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables and recursion stack space
- The space requirement S(P) of any algorithm P may therefore be written as S(P) = c + S_p(instance characteristics) , where c is constant

Time Complexity (Computing Time)

- The time complexity of an algorithm is the amount of computer time it needs to run to completion
- The time T(P) taken by a program P is the sum of the compile time and the run (or execution time)
- The compile time does not depend on the instance characteristics
- Hence, we concern with just the run time of a program
- This run time is denoted by t_p(instance characteristics)

Step Count for Array Element Addition

Statement	s/e	frequency	total steps
1 Algorithm Sum(a,n)	0	–	0
2 {	0	–	0
3 $s := 0.0;$	1	1	1
4 for $i := 1$ to n do	1	$n+1$	$n+1$
5 $s := s + a[i];$	1	n	n
6 return $s;$	1	1	1
7 }	0	–	0
Total			$2n+3$

Step Count for Two Matrix Addition

Statement	s/e	frequency	total steps
1 Algorithm Add(a,b,c,m,n)	0	–	0
2 {	0	–	0
3 for $i := 1$ to m do	1	$m+1$	$m+1$
4 for $j := 1$ to n do	1	$m(n+1)$	$mn+m$
5 $c[i,j] := a[i,j] + b[i,j]$;	1	mn	mn
6 }	0	–	0
Total			$2mn+2m+1$

Brute Force Technique

- Definition: The straight forward method or approach to solve a given problem based on the problem's statement and definitions of the concepts involved is called Brute Force technique.
- It is often easier to implement than a more sophisticated one.
- Because of this simplicity, sometimes it can be more efficient.

Under what conditions the Brute force method is desirable?

- It is useful to solve small-size instances of problem.
- This method is applicable for variety of problems such as finding the sum of n numbers, GCD, power of given number etc.
- We can easily write algorithms for some problems such as sorting, searching, matrix multiplication, string matching etc.
- This can be used as an yard stick with which other alternate problems can be solved and efficient algorithms can be chosen.
- It is simple and can be easily design.

Selection Sort

- In this method, obtain the first smallest number and exchange with the element in the first position. This places the first smallest element in the first place. This completes the first pass.
- In the second pass, obtain the second smallest number from the second position onwards and exchange it with the element in the second position. This places the second smallest element in the second position.
- The process is repeated for remaining elements of the array.

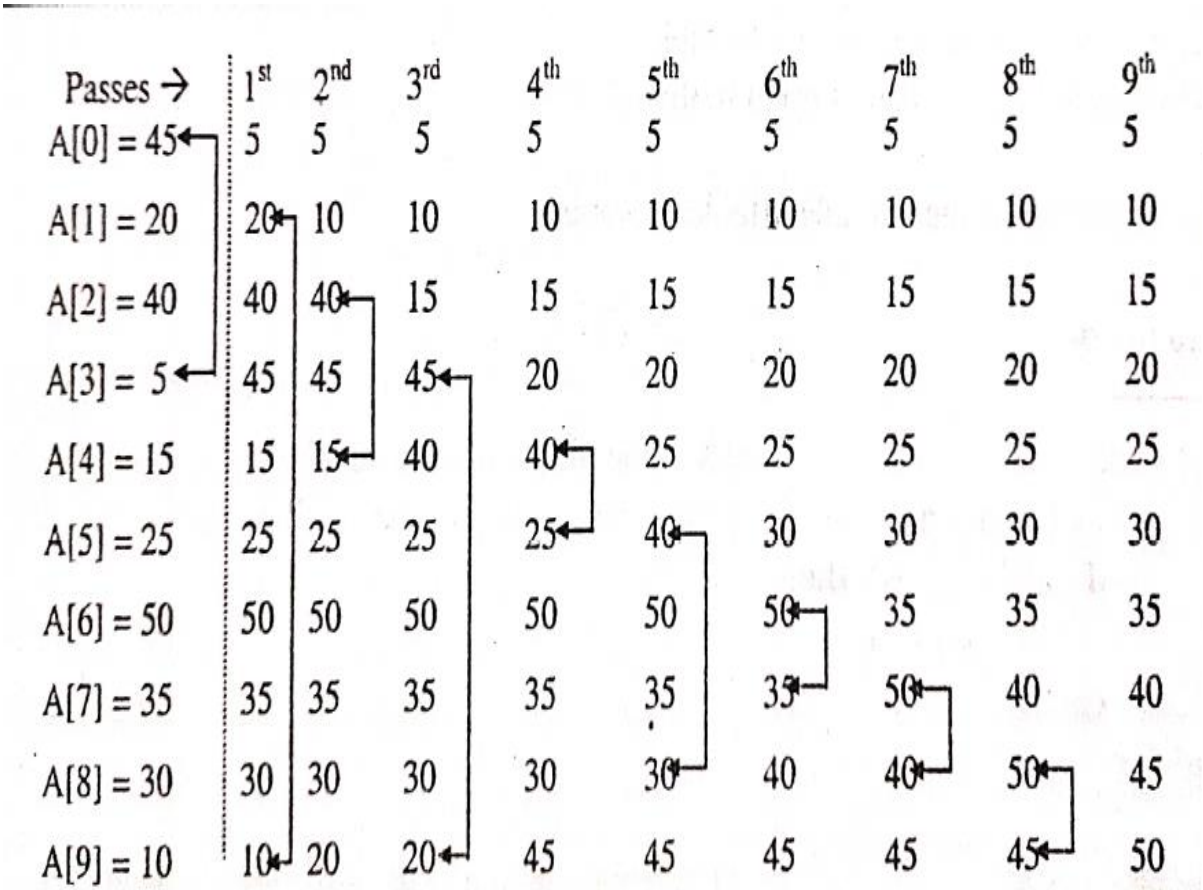


Figure 3.2 Trace for selection sort

ALGORITHM ***SelectionSort(A[0..n − 1])***

{

//Sorts a given array by selection sort

//Input: An array *A[0..n − 1] of orderable elements*

//Output: Array *A[0..n − 1] sorted in nondecreasing order*

for *i ← 0 to n − 2 do*

min ← i

for *j ← i + 1 to n − 1 do*

if *A[j] < A[min]*

min ← j

swap *A[i] and A[min]*

}

Bubble Sort

- Another brute-force application to the sorting problem is to compare adjacent elements of the list and exchange them if they are out of order.
- By doing it repeatedly, we end up “bubbling up” the largest element to the last position on the list.
- The next pass bubbles up the second largest element, and so on, until after n − 1 passes the list is sorted

ALGORITHM BubbleSort(A[0..n − 1])

{

//Sorts a given array by bubble sort

//Input: An array A[0..n − 1] of orderable elements

//Output: Array A[0..n − 1] sorted in nondecreasing order

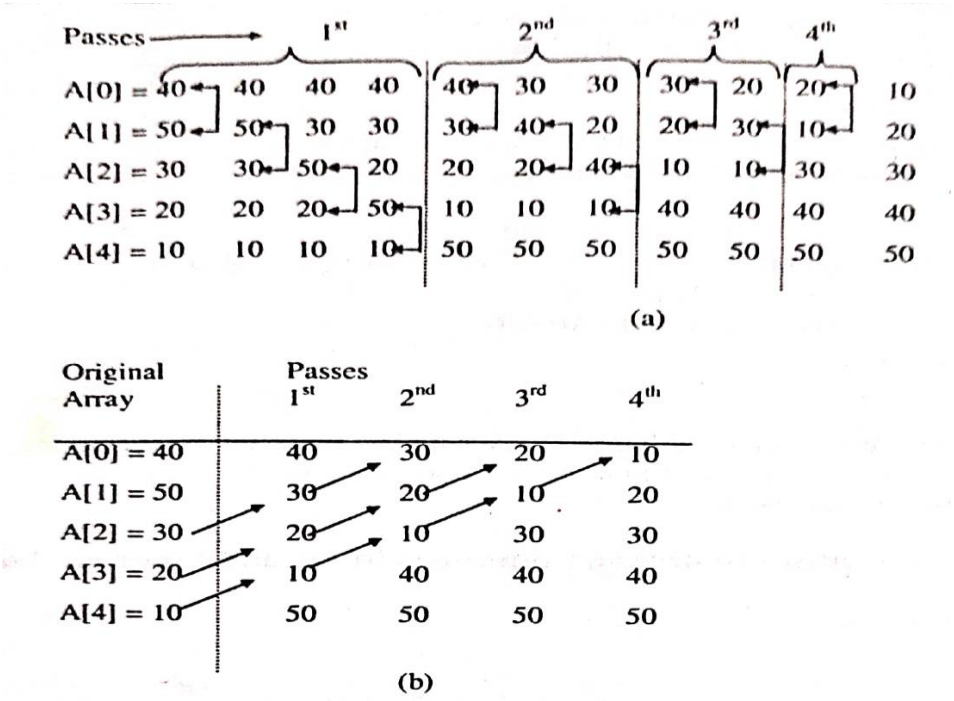
for i ← 0 to n − 2 do

for j ← 0 to n − 2 − i do

if A[j + 1]<A[j]

swap A[j] and A[j + 1]

}



Sequential Search(Linear Search)

- The algorithm simply compares successive elements of a given list with a given search key until either a match is encountered (successful search) or the list is exhausted without finding a match (unsuccessful search).
- A simple extra trick is often employed in implementing sequential search: if we append the search key to the end of the list, the search for the key will have to be successful, and therefore we can eliminate a check for the list's end on each iteration of the algorithm.

```
Algorithm SequentialSearch(A, n, Key)
{
   A[n]←Key
        i←0
        while(a[i]≠Key)
             i←i+1
        end while
        if i < n then
        return i;
        else
        return -1;
        end if
}
```